

Props & State Management in React (Theory)

Welcome to Props & State Management in React! In this session, we transition from static components to dynamic, interactive applications. You will learn how to pass data between components using **Props** and how to manage changing data within a component using the **useState hook**.

1. Passing Data with Props (Properties)

In React, **props** are inputs passed from a parent component to a child component. They allow you to make your components reusable and dynamic.

- **Read-Only Nature:** Props are immutable (read-only) from the perspective of the receiving child component. A child component can never modify its own props.
- **Passing Props:** You pass props inside JSX just like HTML attributes:

```
<PredictionCard title="House Price Estimator" modelType="Regression" />
```

- **Receiving Props:** Inside the child component, accept `props` as a function parameter and access them using dot notation or object destructuring:

```
function PredictionCard({ title, modelType }) {  
  return <h3>{title} ({modelType})</h3>;  
}
```

2. Managing Component Data with State (useState)

While props are passed *down* from above, **state** is data managed *internal* to a component. State represents variables that can change over time—usually in response to user actions (like typing into an input field or clicking a button).

A. What is a Hook?

Hooks are special functions introduced in React 16.8 that let functional components use state and other React features without writing class components. All hooks start with the word `use`.

B. The `useState` Hook

To use state, import it from React and initialize it inside your component function:

```
import { useState } from "react";

function Counter() {
  // count is the state variable, setCount is the function to update it
  // 0 is the initial state value
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

- **Re-rendering:** When you call the state update function (`setCount`), React automatically re-renders the component to display the new state value on the screen.

3. Controlled Components in Forms

For our machine learning dashboard, we need to capture user inputs (e.g., square footage or age) and store them in React state. This is achieved using **Controlled Components**:

- **Binding Value:** The input element's `value` attribute is tied directly to a state variable.
- **Handling Changes:** An `onChange` event listener triggers a state update function every time the user types a character.

```
const [sqft, setSqft] = useState("");

<input
  type="number"
  value={sqft}
  onChange={(e) => setSqft(e.target.value)}
  placeholder="Enter square footage..."
/>
```

4. Reference Links & Further Reading

- **React State: A Component's Memory:** Official guide on declaring and updating state.

- **Passing Props to a Component:** Learn advanced patterns for structuring component hierarchies with props.
-